Docker - Gestión de Redes

1. ¿Por qué necesitamos redes en Docker?

Por defecto, cuando instalamos Docker, cada contenedor se ejecuta en un **entorno aislado**. Esto significa que un contenedor no puede ver ni comunicarse con otro a menos que configuremos explícitamente una vía de comunicación entre ellos.

Las redes en Docker permiten:

- **Aislamiento:** Separar servicios que no deben tener comunicación (por ejemplo, aislar la base de datos de la red pública).
- **Comunicación interna:** Permitir que los contenedores colaboren (ej. que una aplicación Node.js/PHP hable con su base de datos MySQL).
- **Seguridad:** Controlar qué puertos se exponen al exterior (la máquina física del usuario o internet) y cuáles permanecen protegidos en la red interna.

2. Tipos de Redes Principales en Docker

Docker ofrece diferentes controladores de red (*drivers*) según las necesidades de la arquitectura:

Tipo de Red	Descripción y Uso Principal
bridge (Por defecto)	Red interna estándar. Los contenedores conectados a ella pueden comunicarse entre sí, pero **solo mediante sus direcciones IP**, ya que el DNS automático está deshabilitado en la red *bridge* por defecto.
bridge personalizado	**La más recomendada para desarrollo.** Crea una red privada virtual donde Docker activa la resolución DNS interna, permitiendo que los contenedores se comuniquen usando sus **nombres** como si fueran dominios.
host	Elimina el aislamiento de red: el contenedor comparte directamente la pila de red del sistema operativo anfitrión (host). No hay mapeo de puertos (-p).
none	Aislamiento total. El contenedor no tiene interfaces de red externas ni internas (salvo la de *loopback* localhost).

3. Bridge Personalizado. Resolución por Nombre

En entornos profesionales de despliegue web, nunca dependemos de direcciones IP fijas, ya que estas pueden cambiar al reiniciar los contenedores.

Al crear una red personalizada de tipo *bridge*, Docker incorpora un servidor DNS interno. Si un contenedor llamado aplicacion-web necesita conectarse a una base de datos llamada mysql-servidor, la aplicación simplemente puede usar mysql-servidor como el **host** en su cadena de conexión.

4. Comandos Esenciales para la Gestión de Redes

Los alumnos deben dominar los siguientes comandos de la CLI de Docker:

- **Crear una red personalizada:**

docker network create --driver bridge mi-red-personalizada

- **Listar las redes disponibles en el sistema:**

docker network ls

- **Inspeccionar una red (ver qué contenedores están conectados y sus IPs):**

docker network inspect mi-red-personalizada

- **Eliminar una red (debe estar vacía de contenedores):**

docker network rm mi-red-personalizada

5. Ejemplo Práctico: Conectar Nginx con MySQL mediante una Red

Un caso de uso clásico en el módulo es aislar la base de datos y permitir que un servidor web (o una aplicación backend) acceda a ella de forma segura.

Paso 1: Crear la red personalizada

Creamos una red personalizada de nombre **mi-red-daw**

docker network create mi-red-daw

Comprobamos que se haya creado

docker network ls

Paso 2: Arrancar el contenedor de MySQL dentro de la red

Asignamos el nombre **mysql-db** al contenedor y lo conectamos a nuestra red personalizada:

docker run -d --name **mysql-db** --network **mi-red-daw** -p 3306:3306 -e MYSQL_ROOT_PASSWORD=**root1234** -e MYSQL_DATABASE=app_db -v sql:/var/lib/mysql mysql

Paso 3: Arrancar el contenedor web (nginx:alpine) en la misma red

docker run -d --name **servidor-nginx** --network **mi-red-daw** -p 8080:80 nginx:**alpine**

Paso 4: Comprobar la conectividad interna

Podemos verificar que los contenedores se ven entre sí ejecutando un comando de diagnóstico (como ping) desde dentro del contenedor de Nginx hacia el contenedor de la base de datos. La versión de Nginx tiene que ser Alpine porque incluye el comando ping y otras versiones como la latest no lo incluyen.

docker exec -it servidor-nginx ping -c 3 mysql-db

Si el sistema responde con paquetes correctos, significa que la red está perfectamente configurada y aislada del exterior de forma segura.

SI ejecuto el comando al revés, del servidor mysql-db al servidor nginx no funciona porque la imagen mysql que hemos usado no incluye el comando ping.

Práctica de Despliegue y Comunicación de Contenedores en Redes Docker

- **Objetivo:** Configurar una red aislada en Docker para conectar de forma segura un servidor web (Nginx) con una base de datos (MySQL), utilizando volúmenes para garantizar la persistencia de los datos.

Vas a simular el despliegue de una arquitectura web clásica de dos niveles:

1. **Capa de Datos:** Un contenedor de base de datos **MySQL** con un script de inicialización automática y un volumen persistente.
2. **Capa Web:** Un contenedor de **Nginx** que servirá una página estática personalizada para comprobar el estado del despliegue.
3. Ambos contenedores deben estar aislados del exterior pero **conectados entre sí** a través de una red virtual privada de Docker, utilizando resolución de nombres interna.

Paso 1: Crear la estructura del proyecto en tu equipo

Crea una carpeta llamada **practica-redes-daw** en tu ordenador y dentro de ella crea la siguiente estructura de directorios y archivos:

practica-redes-daw/

├── Dockerfile

├── html/

│ └── index.html

└── sql/

└── init.sql

Paso 2: Crear los archivos de contenido

- **html/index.html** (Página principal del servidor web):

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Práctica de Redes Docker - DAW</title>
</head>
<body>
  <h1>¡Conexión de Redes Docker Exitosa!</h1>
```

```
    <p>Módulo de Despliegue de Aplicaciones Web.</p>
</body>
</html>
```

- **sql/init.sql** (Script de inicialización de la base de datos):

```
CREATE TABLE IF NOT EXISTS alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    rol VARCHAR(50) NOT NULL
);
INSERT INTO alumnos (nombre, rol) VALUES ('Profesor DAW', 'Administrador');
INSERT INTO alumnos (nombre, rol) VALUES ('Alumno DAW', 'Desarrollador');
```

- **Dockerfile** (Para empaquetar nuestro servidor Nginx personalizado):

```
FROM nginx:alpine
COPY ./html /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Paso 3: Crear la Red Personalizada en Docker

Para que los contenedores puedan comunicarse utilizando sus nombres de host en lugar de direcciones IP estáticas, creamos una red de tipo ***bridge***:

docker network create **red-practica-daw**

- **Comprobación:** Puedes verificar que la red se ha creado correctamente ejecutando docker network ls

Paso 4: Levantar la Base de Datos (MySQL) en la red

Ejecuta el contenedor de MySQL conectándolo a nuestra red, asignándole el nombre **mysql-db**, la contraseña de root **root1234** y crea un volumen asociado a la carpeta **sql** del proyecto para la persistencia:

docker run -d --name mysql-db --network red-practica-daw -e MYSQL_ROOT_PASSWORD=root1234 -e MYSQL_DATABASE=colegio_db -v vol_mysql_datos:/var/lib/mysql -v "${PWD}/sql:/docker-entrypoint-initdb.d" mysql:8.0

- ¿Qué función cumple el volumen -v "$(pwd)/sql":/docker-entrypoint-initdb.d/? *(Pista: revisa qué ocurre con los scripts .sql al arrancar el contenedor por primera vez).*

Paso 5: Construir y Levantar el Servidor Web (Nginx) en la misma red

Primero, construye tu imagen personalizada de Nginx desde la carpeta del proyecto con el nombre **nginx-practica-redes**:

docker build -t nginx-practica-redes .

A continuación, arranca el contenedor conectándolo a la **misma red** (red-practica-daw) y mapeando el puerto 8080 de tu máquina física al puerto 80 del contenedor:

docker run -d --name **con-nginx-redes** --network red-practica-daw -p 8080:80 nginx-practica-redes

Paso 6: Verificación y Pruebas de Conectividad

Comprueba que todo funciona correctamente respondiendo a los siguientes puntos de entrega:

1. **Prueba Web:** Abre tu navegador y accede a http://localhost:8080. ¿Visualizas la página correctamente?
2. **Prueba de Red Interna (Ping):** Ejecuta el siguiente comando en tu terminal para comprobar que el contenedor web ve al contenedor de la base de datos a través de su nombre:

docker exec -it con-nginx-redes ping -c 3 mysql-db

¿Obtienes respuesta de paquetes? ¿Qué pasaría si intentaras hacer un ping usando un nombre que no está en la misma red?

3. **Prueba de Base de Datos:** Entra de forma interactiva dentro del contenedor de MySQL para comprobar que el script init.sql se ejecutó correctamente y la tabla tiene datos:

docker exec -it mysql-db mysql -u root -proot1234 colegio_db -e "SELECT * FROM alumnos;"

Debe aparecer en pantalla el listado con los registros insertados.

Documentación a entregar. Se debe entregar un documento de nombre **Practica2 - Tu-Nombre** que incluya:

- Capturas de pantalla de la ejecución de los comandos de creación de red y contenedores (docker ps y docker network inspect red-practica-daw).
- El resultado de la prueba de conectividad por ping entre contenedores.
- Una breve explicación de por qué es más seguro y eficiente utilizar una red *bridge* personalizada en lugar de la red por defecto de Docker.